한양대학교
HANYANG UNIVERSITY

Graduation Project Progress Report 1

Title	Building an efficient distributed inference system for open-source LLMs on the cloud
Summary (in 150 words)	As large language models (LLMs) grow in size, running them efficiently has become a major engineering challenge. This project aims to build a scalable and cost-effective LLM inference system using the vLLM framework. We will start by deploying open-source models like Gemma on our local lab server to understand the baseline architecture, memory management, and precision tuning. Once validated, we will containerize the system using Docker and migrate it to a cloud environment. There, we will implement distributed inference across multiple GPUs to optimize throughput and latency. By benchmarking performance and resource costs, this project aligns with the DDPS Lab's focus on cloud optimization. Ultimately, we want to design a practical MLOps pipeline that serves LLMs reliably without excessive infrastructure costs.
Advisor	Kyungyong Lee
Period	2026.03.01 ~ 2026.12.20

Name	Department	Student ID	Hanyang email	Phone
Jihong Min	Data Science Major	2021012660	mjh2856@hanyang.ac.kr	010-2325-2856
Byungyoon Chung	Data Science Major	2023062760	burnix159@hanyang.ac.kr	010-3912-3238

College of Data Science
Hanyang University

1. Progress to Date

- To establish a production-grade, highly scalable, and economically viable distributed inference pipeline for large language models, the research team has systematically executed the initial engineering phases from March 2026 to the present date (June 2026). The accomplished tasks, comprehensive methodologies, and structural milestones are detailed below, showing direct alignment with the DDPS Laboratory's primary research trajectory in cloud resource optimization:

- **Advanced Literature Synthesis & vLLM Core Architecture Mapping (March – April):**

  The team initiated a rigorous academic review of state-of-the-art serving infrastructures, zeroing in on the mathematical and systemic mechanics of the vLLM engine. Specifically, we analyzed the PagedAttention algorithm, which dynamically allocates Key-Value (KV) cache memory into non-contiguous physical blocks mimicking virtual memory operating systems. This deep dive allowed us to understand how to eliminate internal memory fragmentation and maximize batched inference throughput, establishing the theoretical framework necessary for subsequent parameter tuning.

- **Local Testbed Provisioning & Secure SSH Layer Configuration (March – April):**

  To ground our research in empirical data, we secured operational access to the DDPS Laboratory's bare-metal server infrastructure. This involved provisioning a remote development environment by configuring encrypted SSH keys and setting up isolated user spaces. The baseline testbed is configured on a bare-metal lab server equipped with an Intel Xeon E5-1620 v4 processor, 128GB of system memory, and dual NVIDIA Tesla P100 GPUs (16GB VRAM each). This setup provides a hardware-constrained yet scalable local environment, which is crucial for testing initial optimization techniques before migrating to expensive cloud ecosystems..

- **Single-GPU Baseline Benchmarking & Low-Precision Engineering (March – April):**

  Using the local NVIDIA P100 GPU, we successfully initialized and served the Gemma-2b open-source model architecture. Because the older Pascal architecture of the P100 presents strict compute capability limits and lack of native FP16 Tensor Cores, loading the model with default FP32 parameters caused immediate resource exhaustion. To overcome this hardware threshold, we engineered a low-precision adaptation strategy, fine-tuning the framework runtime to utilize half-precision floating-point formats (dtype=half). This established a stable, non-fragmented local baseline with predictable token-generation latency.

- **OpenAI-Compliant RESTful API Server Deployment (March – April):**

  We built and hosted a localized API server layer that inherently complies with standard OpenAI REST API web protocols. By validating asynchronous networking behaviors, token decoding parameters, and multi-turn chat templates, we successfully

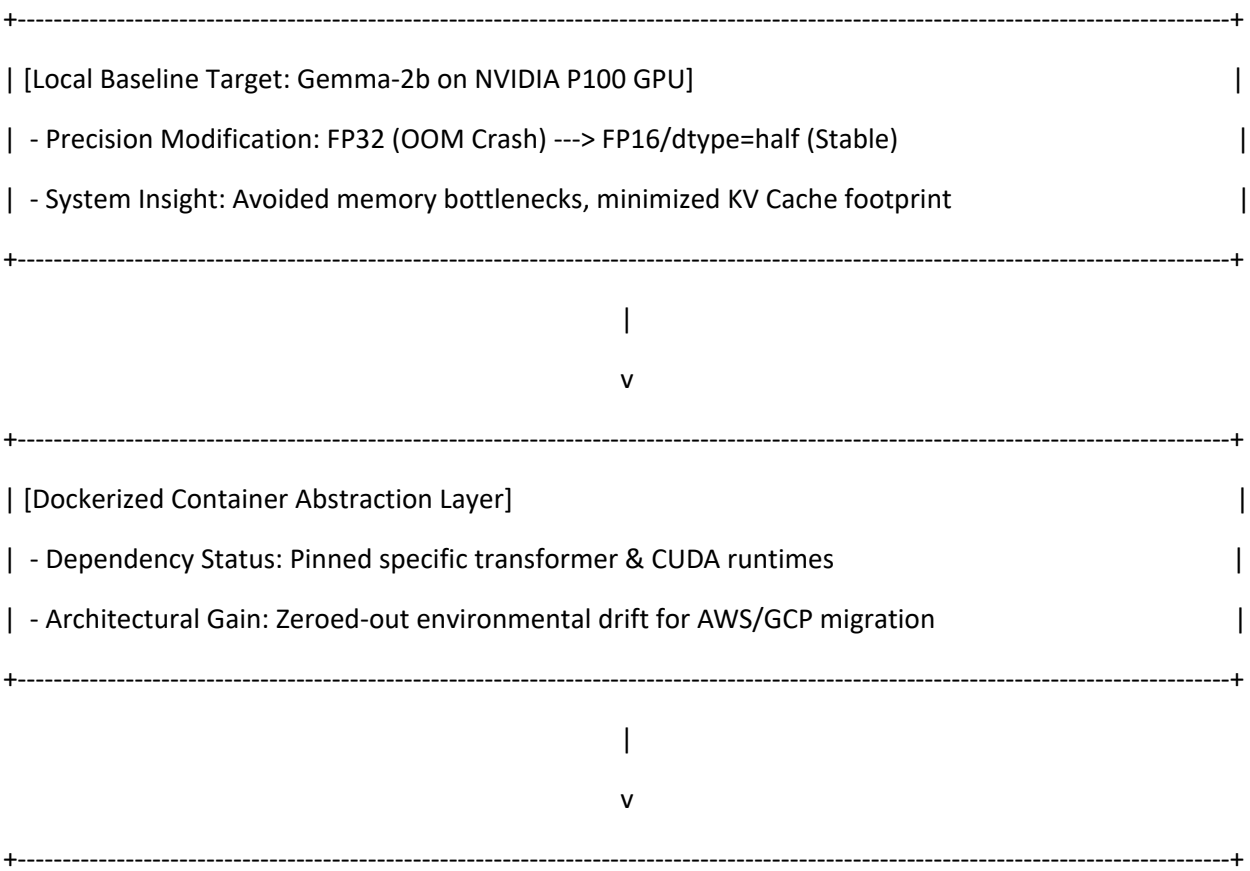
established a consistent interface. This setup guarantees that any downstream application or frontend service can interact with our optimized backend without architectural modifications.

- **Deterministically Packaged Containerization & Dependency Alignment (May – June, In Progress):**

  To ensure the entire vLLM serving stack remains immune to environment drift, version mismatches, or operating system divergence during cloud migration, we initiated structural containerization using Docker. Utilizing official PyTorch base images, we designed custom multi-stage Dockerfiles. During this phase, we encountered and resolved a chain of critical dependency conflicts—most notably a version incompatibility between the transformers library, the specific CUDA runtime wrapper, and the vLLM core compilation layer. Resolving these issues via pinned configurations has granted us an entirely deterministic, cloud-ready deployment artifact.

2. Preliminary Results

- The localized validation loops and containerization experiments have yielded critical empirical data and systemic insights. These preliminary findings actively shape our downstream strategy as we prepare to transition from single-node local hardware to multi-node distributed cloud clusters:



| [Production-Ready Security Integration] |

| - Mechanism: Multi-tenant token-based API Gateway middleware |

| - Future State: Hardened endpoints prepared for distributed cloud load tests |

+----------+

- **Quantifying the Boundary of Low-Precision Execution on Legacy Hardware:**

  Our early empirical data indicates that running unoptimized open-source LLMs natively causes immediate system degradation due to memory bandwidth choking. Enforcing the dtype=half parameter configuration not only prevented Out-Of-Memory (OOM) fatal crashes but also drastically reduced the VRAM allocation required for the base model weights. This resource reduction expanded the memory head-room available for the PagedAttention engine to manage highly concurrent KV caches. This definitive local benchmark will serve as our control variable to calculate the strict acceleration ratios of public cloud instances.

- **Mitigating Environmental Drift via Rigorous Dependency Decoupling:**

  The process of packaging the vLLM environment into a Dockerized architecture revealed that implicit micro-updates in underlying Python libraries often break the tightly coupled C++ extensions within vLLM. By systematically isolating these dependency bugs and establishing a locked package manifest, we achieved architectural decoupling. The resulting Docker image serves as a portable, plug-and-play inference node, ensuring that cloud scaling will not be bottlenecked by infrastructure-level environment errors.

- **Architectural Blueprinting of Token-Gated Production Security:**

  To transition our system from a simple academic prototype into an industry-grade MLOps asset, we designed and integrated a preliminary security middleware layer directly into the containerized API architecture. This layout utilizes token-based API key authentication, restricting endpoint utilization to verified clients. This early integration ensures that when the system is deployed across distributed cloud networks, it is protected against unauthorized inference requests and potential denial-of-service vectors.

3. Plan for Final Submission

- To guarantee the execution of the remaining tasks before the final capstone evaluation, the project timeline has been expanded into specialized workflows. The division of labor is balanced to match our engineering goals: **Min Jihong** takes charge of cloud orchestration, distributed system architecture, and cloud deployment pipelines; **Jeong Byeongyun** spearheads container optimization, API authentication security frameworks, and empirical load testing/benchmarking.

- **June 2026 (Current Phase Consolidation & Refinement):**
    - Finalize validation of the multi-stage Docker image on the local server, ensuring zero runtime degradation under standard API calls.
    - Harden the backend authentication middleware, establishing a robust local testing environment with secured API routing layers.
    - **Mandatory Academic Activity:** Attend the final capstone conference hosted on June 10, 2026, at the FTC 5F Lobby to participate in the poster presentation sessions, gather peer review data, and study advanced architectural variants.
- **July – August 2026 (Cloud Infrastructure Migration & Parallelized Orchestration):**
    - Initialize and configure institutional public cloud spaces (AWS or GCP) by provisioning scalable multi-GPU instances.
    - Deploy the portable Docker containers onto the multi-GPU infrastructure, setting up vLLM's native distributed processing layer.
    - Implement Tensor Parallelism (TP) to split intra-layer weights across adjacent GPUs via NVLink, and Pipeline Parallelism (PP) to segment sequential layers, enabling the system to host larger open-source models with minimized inter-node latency.
- **September – October 2026 (Stress-Testing, Volumetric Load Balancing, & Cost-Modeling):**
    - Execute rigorous stress testing using automated traffic simulators to continuously monitor system performance under fluctuating concurrent user requests.
    - Collect quantitative metrics on transaction latency profiles (Time-To-First-Token, Inter-Token Latency) and cumulative throughput rates (Tokens Per Second).
    - Formulate a structured cost-efficiency model, mapping the financial expenditure of public cloud GPU runtimes against the performance improvements over the local lab server baseline.
- **November 2026 (Bottleneck Eradication & Academic Manuscript Formulation):**

- Analyze the aggregated cloud benchmarking datasets to identify and eliminate systemic distributed bottlenecks, particularly focusing on data-transfer overhead across parallel nodes.
- Draft the comprehensive graduation thesis, ensuring all data-driven findings are structured into a formal manuscript aimed at a peer-reviewed conference on AI Systems or Cloud Computing.

- **December 2026 (Final Deliverable Synthesis & Capstone Dissemination):**
  - Compile all system files, finalize the comprehensive progress reports, and deliver the final capstone project defense presentation.

4. References

- Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., ... & Stoica, I. (2023). "Efficient Memory Management for Large Language Model Serving with PagedAttention." Proceedings of the 29th Symposium on Operating Systems Principles (SOSP '23).
- Blancas, E. (2024). "Deploying VLLM: a Step-by-Step Guide." Ploomber Blog.
- Lee, S., Heo, Y., Park, S., and Lee, K. (2022). "PROFET: PROFiling-based CNN Training Latency ProphET for GPU Cloud Instances." IEEE International Conference on Big Data.
- Choi, J., Park, S., and Lee, K. (2022). "All-You-Can-Inference: Serverless DNN Model Inference Suite." Eighth International Workshop on Serverless Computing held with ACM/IFIP Middleware.

[ACKNOWLEDGMENTS] This paper was supported by Korea Institute for Advancement of Technology (KIAT) grant funded by the Korea Government (Ministry of Education) (P0030395, Advanced Industry Talent Training Bootcamp Program)